

Tema 10:

Genericidad

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Sergio Caverio



Universidad
Rey Juan Carlos

Curso 2025-2026

Contenidos

- Punto de partida
- Definición
- Ejemplo en Java
- Pseudo-genericidad en Pascal
 - TElemento
 - Pila genérica en Pascal – Arrays
 - Pila genérica en Pascal – Memoria dinámica
- Ejercicio propuesto

Punto de partida

Hasta ahora, hemos definido estructuras ligadas a un tipo de dato específico (ej. integer).

```
unit uColaInteger;  
interface  
  
type  
  nodo = record  
    info: integer;  
    sig: ^nodo;  
  end;  
  
  tCola = record  
    first, last: ^nodo;  
  end;  
  
  procedure initialize_queue(var c: tCola);  
  function empty_queue(c: tCola): boolean;  
  function first(c: tCola): integer;  
  function last(c: tCola): integer;  
  procedure enqueue(var c: tCola; x: integer);  
  procedure dequeue(var c: tCola);
```

Punto de partida

- **¿Qué ocurre si el tipo de dato a almacenar cambia?**
- Por ejemplo, si necesitamos una cola de string o de registros complejos:

Punto de partida

- **¿Qué ocurre si el tipo de dato a almacenar cambia?**
 - Si necesitásemos una cola de string o de registros...
 - Deberíamos duplicar todo el código de la unidad.
 - Mantenimiento costoso: un error en la lógica debe corregirse en “N” archivos y en “X” líneas.
 - Crecimiento exponencial de la complejidad del proyecto.
 - Todo ello, teniendo en cuenta que las colas de otros elementos presentan un comportamiento idéntico: solo cambia el tipo base.
- **La solución:** Separar el algoritmo (cómo funciona la cola) del dato (qué guarda la cola).

¿Por qué genéricos?

Reutilización

- Escribir la ED una sola vez y usarla para cualquier tipo de dato.

Seguridad

- El compilador verifica que los tipos coincidan, evitando errores en ejecución.

Abstracción

- Nos permite enfocarnos en la lógica estructural (punteros, enlaces) sin importar el contenido.

Definición

- **Genericidad:** mecanismo utilizado en los lenguajes de programación con tipado estático para especificar el tipo de los elementos de una estructura de datos.
- Permiten programar las operaciones de una ED (o colección de elementos) utilizando como elemento base un tipo genérico no especificado y luego especificar en cada uso según se necesite.
- Debemos **especificar** el **tipo base** de los elementos contenidos **en una ED al utilizarla**.

Ejemplo en Java

- En Java podemos definir TADs genéricos con el marcador `<T>`
- Al definir una clase, usamos un parámetro de tipo (marcador de posición) que se reemplazará más tarde.

```
public class MiLista<T> {  
    private T contenido;  
  
    public void set(T t) {  
        this.contenido = t;  
    }  
  
}
```



```
MiLista<String> lista =  
    new MiLista<>();  
lista.set("Hola");
```

(Pseudo) genericidad en Pascal

- El estándar de Pascal no soporta TADs genéricos nativos por lo que haremos uso del patrón "Tipo Abstracto de Elemento".
- Dividiremos nuestro código en dos responsabilidades:
 - **uTElement**: Define QUÉ guardamos a partir del registro TEElement. Es lo único que cambiaremos según el proyecto.
 - **uTADElement**: Define CÓMO se comporta la estructura. Utiliza TEElement sin saber qué contiene.

No es la solución óptima, pero sí algo más parecido a lo ideal

(Pseudo) genericidad en Pascal

- Compara cómo el nodo de una pila no usa un tipo básico, sino un tipo no detallado aquí y denominado “TElement”.

```
type
  tnodo = record
    info: TElement; { Dato abstracto }
    ant: ^tnodo;
  end;
  tPilaElementos = ^tnodo;

procedure push(var p: tPilaElementos; x: TElement);
procedure peek(p: tPilaElementos, var e TElement;
```

TElemento

- Cada vez que necesitemos pilas de distintos tipos, habrá que definir una nueva unidad TElemento
- Dentro de la unidad TElemento, creamos un nuevo tipo definido que será el tipo base de nuestra pila.
 - Ej. si necesitamos una pila de personas, TElemento exportará el tipo Persona y sus operaciones.
- Las operaciones de asignación y comparación del tipo base deben definirse en la unidad TElemento:
 - `assign(var e: TElemento; e2: TElemento);`
 - `lessThan(e: TElemento; e2: TElemento): boolean;`
 - `equalTo(e: TElemento; e2: TElemento): boolean;`
 - ...

TElemento

```
unit uTElemento;  
  
interface  
  
type  
    TElemento = record  
        { definición de los campos del tipo TElemento }  
    end;  
  
procedure assign(var e: TElemento; e2 : TElemento);  
function lessThan(e: TElemento; e2 : TElemento): boolean;  
function equalTo(e: TElemento; e2 : TElemento): boolean;  
function toString(e: TElemento): string;  
  
{ cualesquiera otras operaciones necesarias }
```

TElemento: Ejemplo

```
unit uTElement;  
  
interface  
  
type  
    TElement = record  
        edad: integer;  
        nombre: string[50];  
    end;  
  
procedure assign(var e: TElement; e2 : TElement);  
function toString(e: TElement): string;
```

TElemento: Ejemplo

implementation

```
procedure assign(var e: TElement; e2 : TElement);  
begin
```

```
    e.edad := e2.edad;
```

```
    e.nombre := e2.nombre;
```

```
end;
```

```
function toString(e: TElement): string;
```

```
begin
```

```
    toString := '(' + IntToStr(e.edad) + ', ' + e.data2 + ')';
```

```
end;
```

```
end.
```

Pila genérica (con arrays)

```
unit uPilaGenerica;  
  
interface  
  
uses uTElemento;  
  
const  MAX_ELEMENTOS = 100;  
  
type  
    tPilaTElemento = record  
        cima: integer;  
        stack: array[1..MAX_ELEMENTOS] of TElemento;  
    end;  
  
procedure initialize(var p: tPilaTElemento);  
procedure push(var p: tPilaTElemento; x: TElemento);  
procedure pop(var p: tPilaTElemento);  
function peek(p: tPilaTElemento, var x: TElemento);  
function isEmpty(p: tPilaTElemento): boolean;
```

Pila genérica (con arrays)

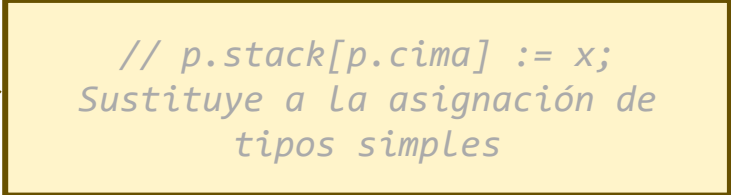
implementation

```
procedure initialize(var p: tPilaTElemento);  
begin  
    p.cima := 0;  
end;
```

```
function isEmpty(p: tPilaTElemento): boolean;  
begin  
    isEmpty := p.cima = 0;  
end;
```

Pila genérica (con arrays)

```
procedure push(var p: tPilaTElemento; x: TElemento);  
begin  
    if p.cima < MAX_ELEMENTOS then  
    begin  
        p.cima := p.cima + 1;  
        assign(p.stack[p.cima], x);  
    end  
end;
```

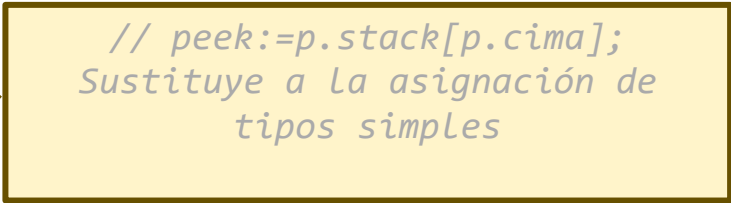


*// p.stack[p.cima] := x;
Sustituye a la asignación de
tipos simples*

Pila genérica (con arrays)

```
procedure pop(var p: tPilaTElemento);  
begin  
    if not isEmpty(p) then  
        p.cima := p.cima - 1;  
end;
```

```
function peek(p: tPilaTElemento, var x: TElemento);  
begin  
    if not isEmpty(p) then  
        assign(x, p.stack[p.cima]);  
end;
```



*// peek:=p.stack[p.cima];
Sustituye a la asignación de
tipos simples*

Pila genérica (con memoria dinámica)

```
unit uPilaGenerica;  
  
interface  
  
uses uTElemento;  { usa la misma unit uTElemento que la implementación  
                    con arrays}  
  
type  
    tnode = record  
        info: TElemento;  
        ant: ^tnode;  
    end;  
    tPilaTElemento = ^tnode;  
  
procedure initialize(var p: tPilaTElemento);  
procedure push(var p: tPilaTElemento; x: TElemento);  
procedure pop(var p: tPilaTElemento);  
function peek(p: tPilaTElemento, var x TElemento);  
function isEmpty(p: tPilaTElemento): boolean;
```

Pila genérica (con memoria dinámica)

implementation

```
procedure initialize(var p: tPilaTElemento);
```

```
begin
```

```
    p := nil;
```

```
end;
```

```
function isEmpty(p: tPilaTElemento): boolean;
```

```
begin
```

```
    isEmpty := p = nil;
```

```
end;
```

Pila genérica (con memoria dinámica)

```
procedure push(var p: tPilaTElemento; x: TElemento);  
var  
    nuevo: ^tnodo;  
begin  
    new(nuevo);  
    assign(nuevo^.info, x); { Sustituye al := para tipos simples }  
    nuevo^.ant := p;  
    p := nuevo;  
end;
```

Pila genérica (con memoria dinámica)

```
procedure pop(var p: tPilaTElemento);  
var  
    aux: ^tnodo;  
begin  
    if not isEmpty(p) then  
    begin  
        aux := p;  
        p := aux^.ant;  
        dispose(aux);  
    end;  
end;
```

Pila genérica (con memoria dinámica)

```
function peek(p: tPilaTElemento, var x: TElemento);  
begin  
    if not isEmpty(p) then  
        assign(x, p^.info);  
end;
```

Funcionalidades adicionales

- Como en el caso de TADs Pila no genéricos, podemos ampliar nuestro TAD Pila genérico con operaciones extra tales como:
 - `is_full()` → Comprueba si está llena
 - `clear()` → Vacía la pila
 - `npop(n)` → Saca n elementos de la cima de la pila
- De nuevo se proponen como ejercicio.

Ejercicio propuesto

- Implementa las ED estudiadas en esta asignatura (Lista, Cola, ArbolBST, Grafo...) para que el elemento a almacenar sea un TElemento.